

# A Visual Overview of the SSTSS-GenSim Tool Workflow

SSTSS-GenSim Tool Demo

The **SSTSS-GenSim Tool** facilitates scenario-based simulation and testing for ADS. SSTSS-GenSim comprises eight modules: (i) Scenario Selection Module, (ii) Scenario Implementation Module, (iii) Scenario Configuration Module, (iv) Simulator and ADS Integration Module, (v) Scenario Execution Module, (vi) Data Collection and Preprocessing Module, (vii) Safety Evaluation Module, and (viii) Data Visualization Module. Each module corresponds to a specific stage of the scenario-based safety testing pipeline, collectively forming a unified testing tool for evaluating the safety performance of ADS. In this tutorial, we provide a demo for using the SSTSS-GenSim tool. This demo provides detailed information on Section 5 of the article.

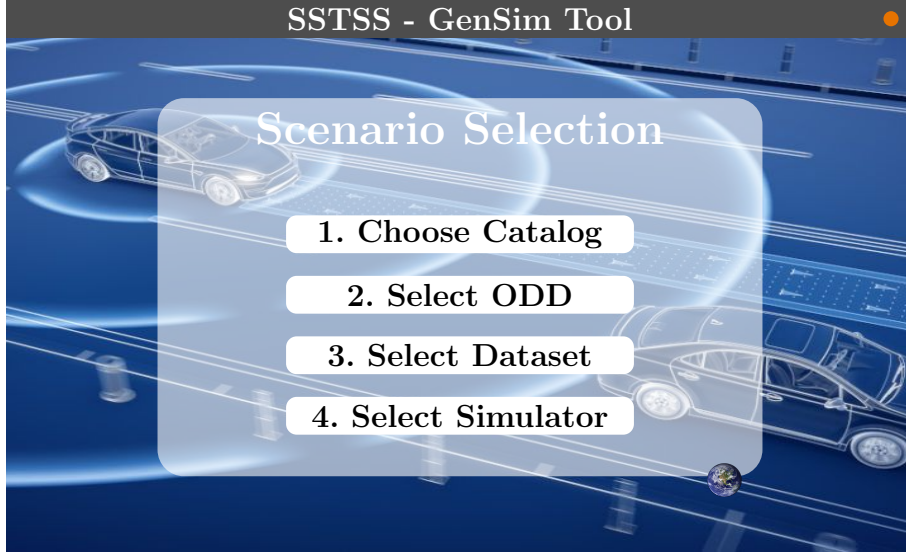


Figure 1: GUI for the Scenario Selection Module

## 1 Scenario Selection Module

For Scenario Selection Module, users provide four inputs.

**Step 1: Choose Catalog:** Select the appropriate scenario catalog. The current version includes two default catalogs: *Singapore* and *NHTSA*. For demonstration, the *Singapore* Catalog is selected. Users can also add new scenarios by clicking the *Add Scenario* button and selecting the catalog to which the scenario should be added. Additionally, users have the option to create custom catalogs.

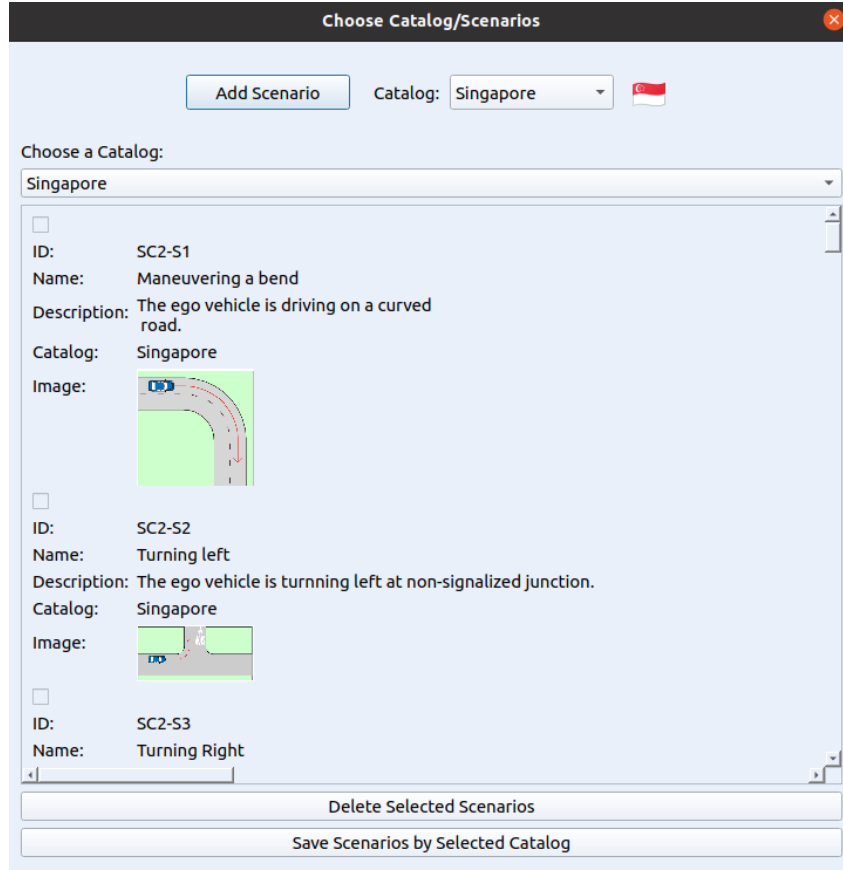
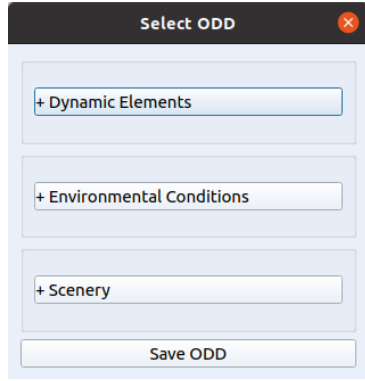


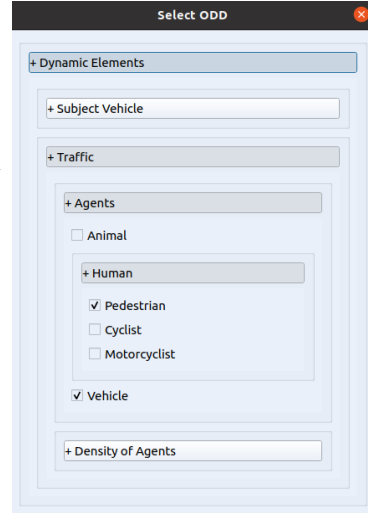
Figure 2: Choose Catalog window

Figure 3: Add Scenario Interface

**Step 2: Select ODD:** Configure the Operational Design Domain (ODD). The ODD setup comprises three modules — *Dynamic Elements*, *Environment*, and *Scenery*. Each tab can be expanded to specify ODD elements. Each tab in the ODD configuration can be expanded to define specific ODD elements. For the demonstration, we select the ODD parameters based on the operational capabilities of the UT-ADS.

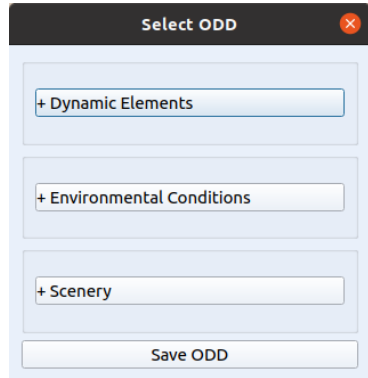


(a) ODD Input

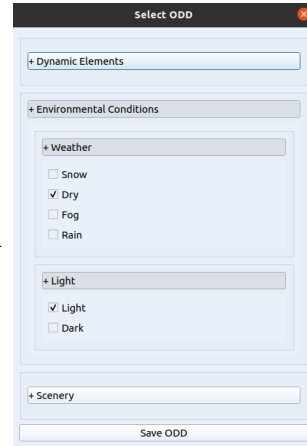


(b) Dynamic Elements

**Dynamic Elements:** Under **Dynamic Elements**, we selected *pedestrians* and *vehicles*, as the UT-ADS is capable of detecting and interacting with both.



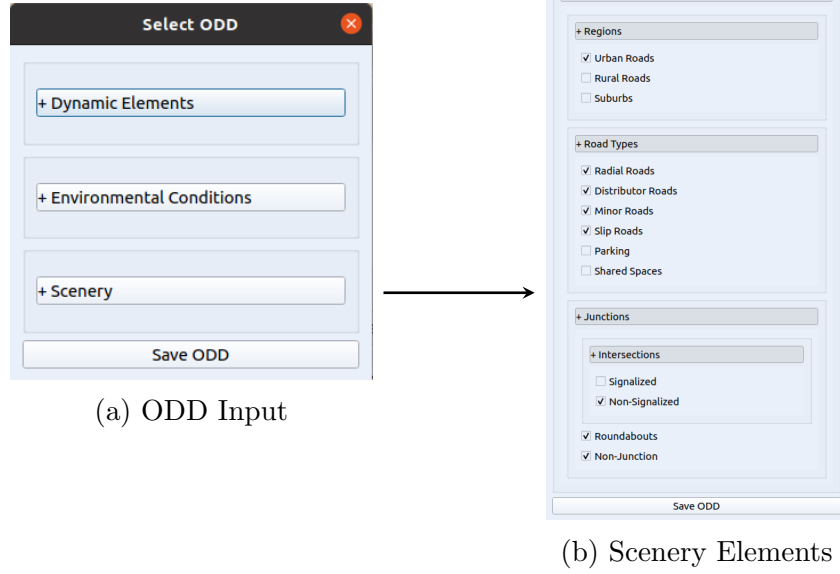
(c) ODD Input



(d) Environmental Conditions

**Environmental Conditions:** We configured the scenario for *daytime* with *dry weather*.

Figure 4: Process flow for ODD configuration across Dynamic Elements and Environmental Conditions.



**Scenery Elements:** We selected an *urban road environment*. The road type was defined as *radial*, including slip roads, minor, and distributed roads.

Figure 5: Process flow for ODD configuration (continued): Scenery Elements.

**Step 3: Select Dataset:** The tool allows users to select a dataset. The dataset is required to prioritize and select test scenarios. Currently, two datasets are supported. For demonstration, the US Accident Dataset is selected.

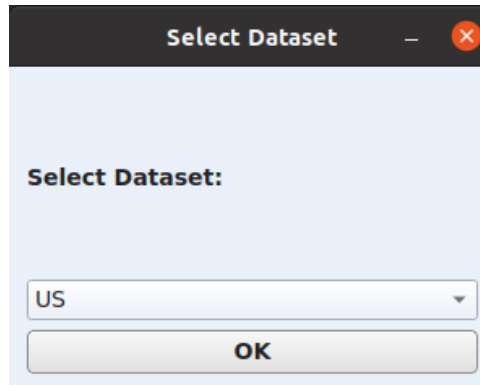


Figure 6: Dataset selection interface

**Step 4: Select Simulator:** The tool allows users to select a simulator; the current version supports CARLA.

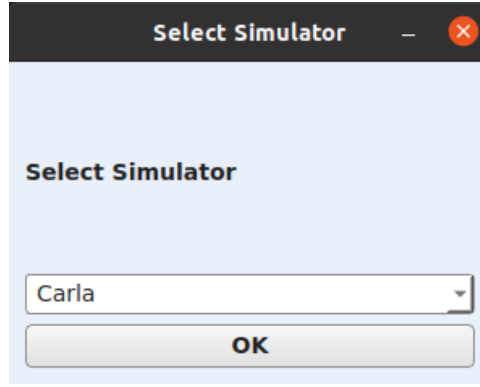


Figure 7: Simulator selection interface

**Final Output:** Upon giving four inputs, the tool generates a structured list of selected scenarios, including their IDs, names, descriptions, and preview images.

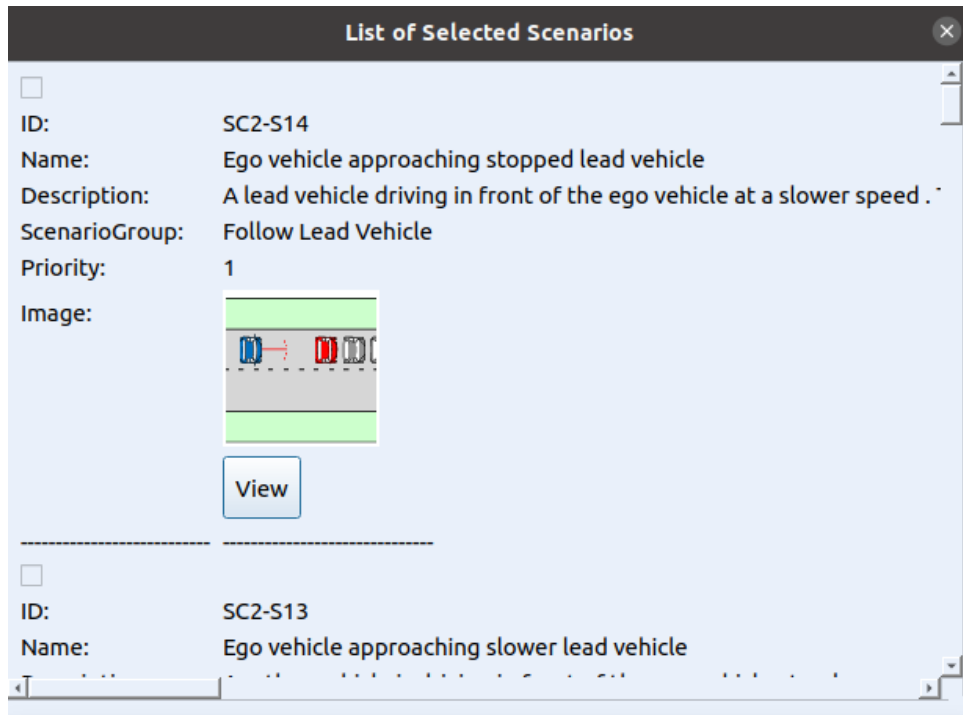


Figure 8: Final list of selected scenarios

## 2 Scenario Implementation Module

Once the scenarios are selected, the tool maps the top-prioritized scenario to a corresponding template, then converts it into an executable `FollowLeadVehicle.py` file. No user interaction is required for this step.

### 3 Scenario Configuration

Selecting *View* opens a configuration window where users can either enter values for input scenario parameters for a single simulation, or users can choose to run the optimizer. For demonstration, we chose an optimizer and specified the minimum and maximum ranges for the input parameters, as shown in the figure below. Note: the demos for running a single simulation are shown on page 9.

| parameter           | min  | max  |
|---------------------|------|------|
| ego_speed           | 10.0 | 50.0 |
| other_vehicle_speed | 10.0 | 50.0 |
| initial_distance    | 2.0  | 20.0 |
| simulation_duration | 90   |      |

| ID | ego_speed | other_vehicle_speed | initial_distance | timeout | loss |
|----|-----------|---------------------|------------------|---------|------|
|----|-----------|---------------------|------------------|---------|------|

Figure 9: Scenario parameter configuration interface

Once the parameters are configured, the tool generates `FollowLeadVehicle.xml` files for the simulator and updates the `FollowLeadVehicle.py` with user-configured parameters, runs the entire pipeline, and based on the TTC feedback, it selects the new values of the input parameter and runs the simulation again. No user interaction is required for this step.

### 4 Simulator and ADS Integration Module

Upon pressing *Run Optimizer*, the CARLA starts, then the ADS stack, and then the scenario runner. The environment is set up, and the actors are initialized.

## 5 Scenario Execution

The driving path is configured for the ADS in green colour, and the scenario starts execution. The system visualizes both the CARLA simulation view and the RViz perception view.



(a) CARLA view

(b) RViz view

Figure 10: Scenario execution in CARLA and RViz

After the simulation is completed, the system automatically generates the file `FollowLeadVehicle.log`. This step does not require any user interaction.

## 6 Data Collection and Preprocessing

Once the scenario execution is finished, the system converts the `FollowLeadVehicle.log` file into `FollowLeadVehicle_data.csv`. Invalid or incomplete files are automatically discarded during this process. No user interaction is required.

## 7 Safety Evaluation Module

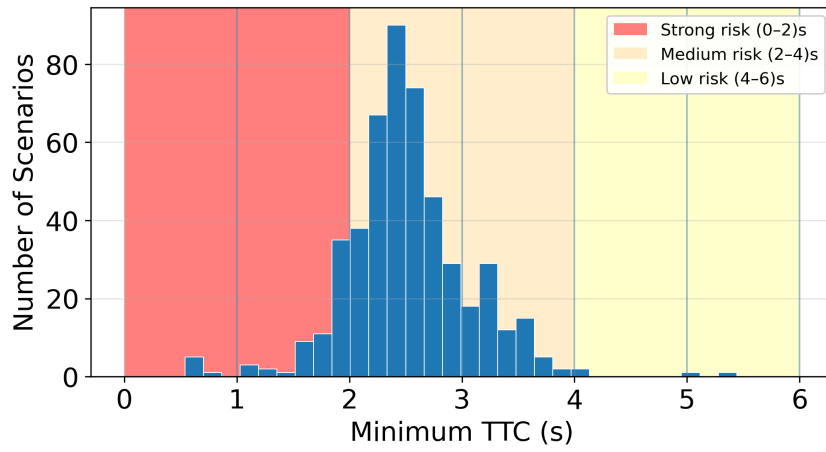
The Safety Evaluation Module computes three safety metrics:

- Time-to-Collision (TTC),
- RSS Safe Distance, and
- Jerk.

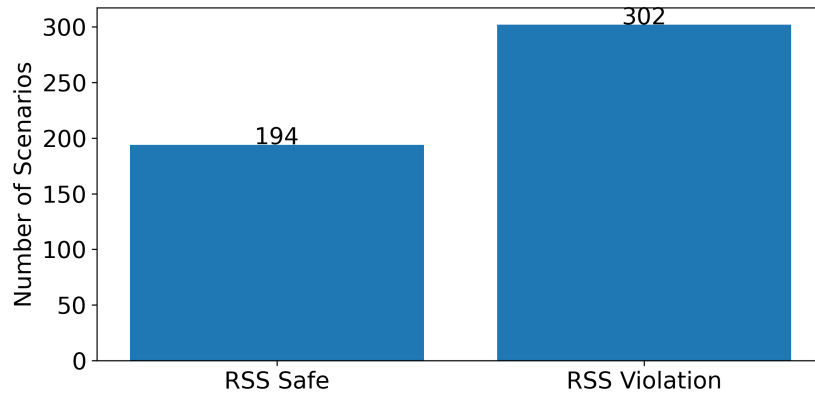
The computed metrics are saved in `FollowLeadVehicle_metrics.csv`. This step is fully automated and does not require user input.

## 8 Data Visualization

After all simulations are completed, selecting *Show Results* displays the safety metric graphs as in Figure 11.



(a) Distribution of Minimum TTC



(b) Distribution of RSS safe Distance Compliance

Figure 11: Plots for TCC and RSS Violations



## Demo for Single Simulation run

At the scenario configuration stage, if the user chooses manual configuration, the same pipeline described above is executed. The data visualization module will generate a single figure per simulation showing the safety metrics for each run.

For example, if user input the ego vehicle speed  $v_r = 40$  km/h, the lead vehicle speed  $v_f = 40$  km/h, and the initial distance  $d_i = 25$  m. Two plots will be generated as shown in Figure 12a and Figure 12b.

Figure 12a shows the plot for the RSS safe distance safety metric. The plot shows the speed of the ego and lead vehicle, the actual distance  $d$  between the ego and lead vehicle, and the theoretical RSS safety distance  $d'_{\min}$  over time. The red dashed line shows the theoretical RSS minimum safe distance  $d'_{\min}$  that the ego vehicle should maintain. The red solid line shows the distance  $d$  between the lead vehicle and the ego vehicle. The ego vehicle maintains a safe following distance if the solid line remains above the dashed line. If the solid line drops below the dashed line, the ego vehicle is not maintaining sufficient distance according to the RSS model. Figure 12b shows the jerk at each time stamp.

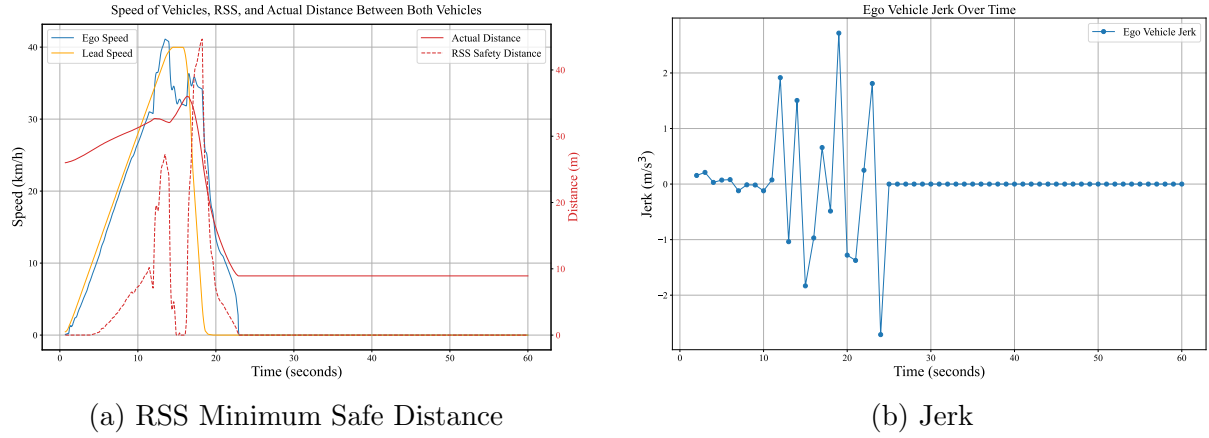


Figure 12: Visualization of simulation results for a Manual Configuration showing (a) RSS safe distance, (b) jerk.